Shift V2 - Cairo

Security Review

April 6, 2026 - April 11, 2026

Contents

1. About SBSecurity 3

2. Disclaimer 3

3. Risk classification 3

4. Executive Summary 4

5. Findings 5

Medium severity 7

[M-01] Incorrect maintenance fee collection period 7
[M-02] A Malicious user can grief fast redeems by occupying the cap till cutoff_epoch and re-requesting before lock_epoch 9

Low severity 10

[L-01] request_fast_redeem_status does not account for s_min_redeem resulting in an inaccurate status 10
[L-02] mint partial fill consumes deposits without minting full requested shares 12
[L-03] OFT _debit ignores min_amount slippage guard 14

Informational severity 16

[I-01] Off-by-one history range checks 16
[I-02] Per-Vault Operator Approval Lacks Batch or Global Alternative 17
[I-03] _settle_redeem_batch_internal has an unreachable else branch 18
[I-04] Possible rounding error in assets and shares calculation during mint 19
[I-05] s_min_redeem can be enforced in redeem/withdraw functions to prevent users from holding dust amounts in s_pending_redeems and possible dust accumulation in pool 20
[I-06] set_fast_redeem_fee_bps allows 100% fee, leading to division by zero in withdraw and total fund loss in redeem 21
[I-07] MAX_DEPOSIT_CLAIM_BATCH Can Cause Incomplete Share Minting 23
[I-08] claimable_fast_redeem_request Returns An _assets Value That withdraw Cannot Accept 24

Gas optimization 27

[G-01] Redundant Storage Read in Deposit Claim Loop 27

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **SHIFT-NebulaLabs/shift-core-v2-cairo** repository was conducted by **SBSecurity**. The review was performed by **MonteCristo**, **Codex, blckhv, Velev, Slavcheww** who collectively identified **14 issues** in total.

Overview

Project	Shift V2 - Cairo
Commit Hash	7212979a34738e7bc214eeef6fa00f56d04e55a7
Resolution	690fa8d65dadc46e487cfa56b25f1cb891a1f686
Timeline	April 6, 2026 - April 11, 2026

Scope

sy_vault/vault.cairo, sy_vault/redeem.cairo, sy_vault/epoch.cairo, sy_vault/base.cairo, sy_vault/deposit.cairo, sy_vault/math.cairo, hub.cairo, admin_gateway.cairo, address_registry.cairo, liquidity_pool.cairo, sy_share.cairo, crosschain/sy_oft_adapter.cairo, crosschain/sy_share_peer.cairo, types/constants.cairo, types/errors.cairo, types/types.cairo

Post Audit Condition

The codebase demonstrates solid development practices. While not strictly necessary, an additional audit is recommended to further strengthen confidence in the system's security posture.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	3
Informational Risk	8
Gas Optimization	1

ID	Title	Severity	Resolution
[M-01]	Incorrect maintenance fee collection period	Medium	**Fixed**
[M-02]	A Malicious user can grief fast redeems by occupying the cap till `cutoff_epoch` and re-requesting before `lock_epoch`	Medium	**Fixed**
[L-01]	`request_fast_redeem_status` does not account for `s_min_redeem` resulting in an inaccurate status	Low	**Fixed**
[L-02]	`mint` partial fill consumes deposits without minting full requested shares	Low	**Fixed**
[L-03]	OFT `_debit` ignores `min_amount` slippage guard	Low	**Fixed**
[I-01]	Off-by-one history range checks	Info	**Fixed**
[I-02]	Per-Vault Operator Approval Lacks Batch or Global Alternative	Info	Acknowledged
[I-03]	`_settle_redeem_batch_internal` has an unreachable `else` branch	Info	**Fixed**
[I-04]	Possible rounding error in assets and shares calculation during mint	Info	**Fixed**

[I-05]	`s_min_redeem` can be enforced in `redeem`/`withdraw` functions to prevent users from holding dust amounts in `s_pending_redeems` and possible dust accumulation in pool	Info	**Fixed**
[I-06]	`set_fast_redeem_fee_bps` allows 100% fee, leading to division by zero in `withdraw` and total fund loss in `redeem`	Info	**Fixed**
[I-07]	`MAX_DEPOSIT_CLAIM_BATCH` Can Cause Incomplete Share Minting	Info	**Fixed**
[I-08]	`claimable_fast_redeem_request` Returns An `_assets` Value That `withdraw` Cannot Accept	Info	**Fixed**
[G-01]	Redundant Storage Read in Deposit Claim Loop	Gas	**Fixed**

Medium severity

[M-01] Incorrect maintenance fee collection period

Severity: Medium Risk

Description: Maintenance fee deduction logic differs from the solidity version.

Solidity contract always collects fee for `s_maintenanceFeeInterval`:

```
if (s_maintenanceFeeInterval > 0 && s_maintenanceFeeBps > 0
    && (_epoch - s_lastFeeCollectionEpoch >= s_maintenanceFeeInterval))
  {
      uint256 priceBefore  = _price;
@>    uint256 feeDeduction = (_price * s_maintenanceFeeBps * s_maintenanceFeeInterval)
```

Cairo contract collects fee for elapsed epochs since the last fee collection

```
let epochs_since = if _epoch > last {
      _epoch - last
  } else {
      0
  };
  if epochs_since >= maint_interval {
      let deduction = sy_vault_lib::compute_maintenance_fee_deduction(
@>        price, maint_bps, epochs_since.into(),
      );
```

Impact:

Cairo contract may collects unfairly higher maintenance fee during the initial phase of the protocol:

- `s_maintenance_fee_bps = 0` on deployment
- Initial epoch is settled
- During a few epochs, initial shares are minted
- Admin sets `s_maintenance_fee_bps > 0`, `s_maintenance_fee_interval = 1`
- A new epoch is settled
- `epochs_since > 1` and maintenance fee will be collected for multiple epochs
- However, the correct fee collection period should be 1 epoch

Recommendation:

```diff
diff --git a/src/sy_vault/epoch.cairo b/src/sy_vault/epoch.cairo
index 90b2016..df3b7b4 100644
--- a/src/sy_vault/epoch.cairo
+++ b/src/sy_vault/epoch.cairo
@@ -179,7 +179,7 @@ pub mod SyVaultEpochComponent {
    };
    if epochs_since >= maint_interval {
        let deduction = sy_vault_lib::compute_maintenance_fee_deduction(
-          price, maint_bps, epochs_since.into(),
+          price, maint_bps, maint_interval.into(),
        );
        // Defense-in-depth: setter validation ensures deduction < price under valid
        // config. Recovery: call epochSettle(0) to reuse previous price if this panics.
```

Resolution: Fixed

[M-02] A Malicious user can grief fast redeems by occupying the cap till `cutoff_epoch` and re-requesting before `lock_epoch`

Severity: Medium Risk

Description: The `request_fast_redeem` function in the `vault` enforces a check to ensure that any new redeem request does not exceed `cap_shares`. If the cap is exceeded, the transaction reverts.

This design enables a Malicious user to indefinitely block the redeem cap up to `cycle.cutoff_epoch`. The attacker can then cancel their request and immediately submit a new fast redeem request before `cycle.lock_epoch`. As a result, legitimate users are unable to submit fast redeem requests, and the admin is also unable to call `settle_redeem_batch` for that since the `lock_epoch` for the current cycle has not yet been reached.

Attack Scenario:

- Assume total supply is `100e18`

- `cap_bps = 10%`, so `cap_shares = 10e18`

1. A malicious user submits a fast redeem request for `10e18`, fully consuming the cap.

2. Legitimate users attempt to request fast redeem, but all transactions revert due to the cap being reached.

3. Once `cycle.cutoff_epoch` is crossed (epoch 12), users still cannot submit new fast redeem requests because the previous cycle has not yet been settled.

4. Just before `cycle.lock_epoch` (i.e, just before epoch 20), the malicious user:

- Cancels their existing fast redeem request

- Immediately submits a new fast redeem request within a single transaction

5. Since the cutoff has already passed and `cycle.shares == 0` after cancellation, the new request is recorded in the next cycle, effectively consuming the cap for the next cycle too.

6. The attacker can repeat this process indefinitely, continuously blocking the fast redeem mechanism for all other users.

Impact:

The fast redeem mechanism can be abused to cause a persistent denial-of-service for legitimate users.

Recommendation: Consider removing cancellation feature for Fast Redeem Requests to Mitigate this issue

Resolution: Fixed

Low severity

[L-01] `request_fast_redeem_status` does not account for `s_min_redeem` resulting in an inaccurate status

Severity: Low Risk

Description: The `vault::request_fast_redeem_status` function currently checks whether `cycle.shares` is greater than or equal to `cap_shares` and returns `FastRedeemRequestStatus::Ok` if the cap has not been reached.

This function will incorrectly report `FastRedeemRequestStatus::Ok` when `cap_shares - cycle.shares < s_min_redeem`, even though the actual status should be `FastRedeemRequestStatus::CapExceeded`, since users are not allowed to redeem less than `s_min_redeem` shares.

Impact:

`request_fast_redeem_status` will return incorrect status when `cap_shares - cycle.shares < s_min_redeem`

Proof Of Concept:

Paste the PoC below in `sy_vault_fast_redeem_test.cairo` and run it using the following command `snforge test test_fast_redeemStatus_Inaccurate`, also read the comments in the PoC for more details

```
#[test]
#[should_panic(expected: ('SyVault: fast redeem cap hit',))]
fn test_fast_redeemStatus_Inaccurate() {
    let suite = deploy_suite();
    let alice_shares = give_shares(@suite, ALICE(), 100_000_000_000_000_000_000_u256);
    let bob_shares = give_shares(@suite, BOB(), 100_000_000_000_000_000_000_u256);

    start_cheat_caller_address(suite.vault.contract_address, ADMIN());
    suite.vault.set_fast_redeem_cap_bps(1000); // 10%

    suite.vault.set_min_redeem(1_000_000_000_000_000_000_u256); // 1e18
    stop_cheat_caller_address(suite.vault.contract_address);

    // Alice takes full of cap
    let alice_request = (alice_shares * 20 / 100) - suite.vault.min_redeem() + 1_u256 ; // total available
cap is min_redeem - 1
    let cycle_id = request_fast_redeem(@suite, ALICE(), alice_request);
    //Bob's status returns OK

     assert(
        suite.vault.request_fast_redeem_status(BOB()) == FastRedeemRequestStatus::Ok,
        'status is CapExceeded',
    );
     // Bob tries to request but his request reverts as the available cap is less   than minRedeem


    let bob_request = suite.vault.min_redeem()  ;
    let bob_cycle = request_fast_redeem(@suite, BOB(), bob_request);

}
```

Recommendation: In the `request_fast_redeem_status` function make the following changes:

```
fn request_fast_redeem_status(
            self: @ContractState, _controller: ContractAddress,
        ) -> FastRedeemRequestStatus {

               ...
               ..
               .

               if cap_bps < BASIS_POINTS && fast_cycle_id != NO_ACTIVE_CYCLE {

                ...
                ..
                .

-                  if cycle.shares  >= cap_shares {
-                    return FastRedeemRequestStatus::CapExceeded;
-                }

+                 if cycle.shares + self.Base.s_min_redeem.read() >= cap_shares {
+                     return FastRedeemRequestStatus::CapExceeded;
+                 }

            }
            FastRedeemRequestStatus::Ok
        }
```

Resolution: Fixed

[L-02] `mint` partial fill consumes deposits without minting full requested shares

Severity: Low Risk

Description: In the Solidity reference implementation, `mint()` is all-or-nothing: if the requested shares cannot be fully filled from settled deposits, `_consumeDepositEpochsByShares` sets `assetsOut = 0` and `mint()` reverts with `SyVault__NoClaimableDeposit`. Crucially, the revert rolls back all state mutations made during the consume loop.

```
// syVault.sol:341
if (sharesRemaining > 0) assetsOut = 0; // signal incomplete fill

// syVault.sol:231
if (assets == 0) revert SyVault__NoClaimableDeposit();
```

In the Cairo implementation, `_consume_deposit_epochs_by_shares` commits state changes (decrements `s_pending_deposits`, advances `s_deposit_claim_index`) during iteration. If only a partial fill is possible (e.g., 80 out of 100 requested shares), the function returns `(80, consumed_assets)`. Then `mint()` checks `if actual_shares == 0` — which is false — so it proceeds to mint the partial amount without reverting.

```
// vault.cairo:338-344
let (actual_shares, consumed_assets) = self
    .Deposit
    ._consume_deposit_epochs_by_shares(_controller, _shares);

if actual_shares == 0 {
    self.reentrancy_guard.end();
    return 0;  // only triggers when NOTHING is claimable
}
// ... mints actual_shares (may be < _shares)
```

Impact:

A caller requesting `mint(100 shares)` with only 80 shares worth of settled deposits will receive 80 shares instead of a revert. The consumed assets are permanently deducted from the controller's pending deposits. Integrators expecting exact-fill semantics (as per the Solidity behavior) may not check the return value, resulting in fewer shares than expected without any error signal.

Proof Of Concept:

1. Controller has settled deposits worth 80 shares across settled epochs

2. Caller invokes `Hub.mint(vault_id, 100, receiver, controller)`

3. `_consume_deposit_epochs_by_shares(controller, 100)` consumes all 80 shares worth of assets, writes updated `s_pending_deposits` and `s_deposit_claim_index`

4. Returns `(80, consumed_assets)` — `actual_shares = 80 > 0`

5. `mint()` mints 80 shares to receiver, returns `consumed_assets`

6. No revert - caller received 80 instead of 100 with no error

Recommendation: Add a check that the full requested amount was filled, matching Solidity semantics:

```
let (actual_shares, consumed_assets) = self
    .Deposit
    ._consume_deposit_epochs_by_shares(_controller, _shares);

assert(actual_shares == _shares, SYVAULT_NO_CLAIMABLE_DEPOSIT); // revert on partial fill
```

Resolution: Fixed

[L-03] OFT `_debit` ignores `min_amount` slippage guard

Severity: Low Risk

Description: Both OFT `_debit` implementations accept a `min_amount` parameter but never enforce it. The Solidity reference delegates to `super._debit()` which enforces `amount_sent_ld >= _minAmountLD` after dust removal. Cairo performs the lock directly and ignores `min_amount` entirely.

Solidity:

```
// syOFTAdapter.sol:70-77
function _debit(address _from, uint256 _amountLD, uint256 _minAmountLD, uint32 _dstEid)
    internal override returns (uint256 amountSentLD, uint256 amountReceivedLD)
{
    _outflow(_dstEid, _removeDust(_amountLD));
    return super._debit(_from, _amountLD, _minAmountLD, _dstEid);  // super enforces _minAmountLD
}
```

Cairo:

```
// sy_oft_adapter.cairo:153-170
fn _debit(ref self: ..., from: ContractAddress, amount: u256,
    min_amount: u256,  // accepted but NEVER checked
    dst_eid: u32,
) -> OFTDebit {
    let amount_sent_ld = self._remove_dust(amount);
    // ... rate limit + transfer ...
    OFTDebit { amount_sent_ld, amount_received_ld: amount_sent_ld }  // no min_amount check
}
```

The same issue exists in `SySharePeer._debit` on the remote chain, where `min_amount` is similarly accepted but never checked — shares are burned after dust removal with no slippage guard:

```
// sy_share_peer.cairo:126-139
fn _debit(ref self: ..., from: ContractAddress, amount: u256,
    min_amount: u256,  // accepted but NEVER checked
    dst_eid: u32,
) -> OFTDebit {
    let amount_sent_ld = self._remove_dust(amount);
    let mut contract = self.get_contract_mut();
    contract.erc20.burn(from, amount_sent_ld);
    OFTDebit { amount_sent_ld, amount_received_ld: amount_sent_ld }  // no min check
}
```

`_remove_dust()` truncates the amount by `10^(local_decimals - shared_decimals)`. With `shared_decimals = 6` and `local_decimals = 18`, maximum dust removed per transaction is `10^12 - 1` wei.

Impact:

Users specifying `min_amount > 0` as slippage protection when bridging shares have no actual protection. The maximum dust loss per send is approximately `$0.001` at `$1,000` per token — economically negligible per transaction. However, integrators or aggregators composing multiple cross-chain operations that rely on the slippage check as a hard invariant will silently pass when they should revert. This affects both `SyOFTAdapter` (home chain) and `SySharePeer` (remote chain).

Proof Of Concept:

1. User calls `send()` with `amount = 1.000000999999999999e18` and `min_amount = 1.000001e18`

2. `_remove_dust()` truncates to `1.000000000000000000e18` (loses `999999999999` wei)

3. `amount_sent_ld = 1e18` which is less than `min_amount = 1.000001e18`

4. Cairo: proceeds silently, locks `1e18`, no revert

5. Solidity: `super._debit` checks `1e18 >= 1.000001e18` → reverts

Recommendation: Add the missing assertion after dust removal in both `_debit` implementations:

```
let amount_sent_ld = self._remove_dust(amount);
+ assert(amount_sent_ld >= min_amount, 'OFT: amount < min');
```

Resolution: Fixed

Informational severity

[I-01] Off-by-one history range checks

Severity: Information Risk

Description: `SyVault::get_share_price_history` and `SyVault::get_hwm_history` allows `_to_epoch - _from_epoch == MAX_HISTORY_RANGE`.

```
assert(_to_epoch - _from_epoch <= MAX_HISTORY_RANGE, SYVAULT_RANGE_TOO_LARGE);
```

This means return epoch length can be up to 721 instead of 720

Impact:

The first and last returned elements will have the same storage slot modulo 720, so the response will have duplicated items.

Recommendation:

```
diff --git a/src/sy_vault/vault.cairo b/src/sy_vault/vault.cairo
index d797435..3508c36 100644
--- a/src/sy_vault/vault.cairo
+++ b/src/sy_vault/vault.cairo
@@ -768,7 +768,7 @@ pub mod SyVault {
            self: @ContractState, _from_epoch: u64, _to_epoch: u64,
        ) -> Array<SharePriceSnapshot> {
            assert(_to_epoch >= _from_epoch, SYVAULT_INVALID_RANGE);
-           assert(_to_epoch - _from_epoch <= MAX_HISTORY_RANGE, SYVAULT_RANGE_TOO_LARGE);
+           assert(_to_epoch - _from_epoch < MAX_HISTORY_RANGE, SYVAULT_RANGE_TOO_LARGE);

            let mut result: Array<SharePriceSnapshot> = array![];
            let mut i = _from_epoch;
@@ -787,7 +787,7 @@ pub mod SyVault {
            self: @ContractState, _from_epoch: u64, _to_epoch: u64,
        ) -> Array<u256> {
            assert(_to_epoch >= _from_epoch, SYVAULT_INVALID_RANGE);
-           assert(_to_epoch - _from_epoch <= MAX_HISTORY_RANGE, SYVAULT_RANGE_TOO_LARGE);
+           assert(_to_epoch - _from_epoch < MAX_HISTORY_RANGE, SYVAULT_RANGE_TOO_LARGE);

            let mut result: Array<u256> = array![];
            let mut i = _from_epoch;
```

Resolution: Fixed

[I-02] Per-Vault Operator Approval Lacks Batch or Global Alternative

Severity: Information Risk

Description: `Hub::set_operator` only sets operator approval for a single vault at a time. The operator state is stored in each `SyVault`'s `s_operators` map, scoped to `(_controller, _operator)`. There is no mechanism to approve an operator across all vaults in a single call.

[hub.cairo#L158-L168]

```
fn set_operator(
    ref self: ContractState,
    _vault_id: u256,
    _operator: ContractAddress,
    _approved: bool,
) -> bool {
    let caller = get_caller_address();
    let vault = self._get_sy_vault(_vault_id);
    vault.set_operator(caller, _operator, _approved);
    true
}
```

A user with positions in N vaults who wants to delegate management to one operator must submit N separate transactions, one per `_vault_id`.

Impact:

Users who rely on operators (e.g. multisigs, portfolio managers, smart contract wallets) to manage positions across multiple vaults face increased gas cost and operational friction. A forgotten approval on one vault silently blocks the operator from acting on that vault's positions, which could delay time-sensitive claims or redemptions.

Recommendation:

Add a batch variant on the Hub that iterates over a list of vault IDs and applies the same operator approval to each:

```
fn set_operator_batch(
    ref self: ContractState,
    _vault_ids: Span<u256>,
    _operator: ContractAddress,
    _approved: bool,
) {
    let caller = get_caller_address();
    let mut i: u32 = 0;
    while i < _vault_ids.len() {
        let vault = self._get_sy_vault(*_vault_ids.at(i));
        vault.set_operator(caller, _operator, _approved);
        i += 1;
    };
}
```

Same helper function can be added to the Solidity version as well.

Resolution: Acknowledged

[I-03] `_settle_redeem_batch_internal` has an unreachable `else` branch

Severity: Information Risk

Description: The function `redeem::_settle_redeem_batch_internal` contains the following logic:

```
fn _settle_redeem_batch_internal(
            ref self: ComponentState<TContractState>, _cycle_id: u256, _returned_amount: u256,
        ) {
            let mut cycle = self.s_redeem_cycles.read(_cycle_id);
            let cycle_shares = cycle.shares;
    ----@>  assert(cycle_shares > 0, SYVAULT_NO_PENDING_REQUEST);
            assert(!cycle.settled, SYVAULT_CYCLE_ALREADY_SETTLED);

            ...
            ..
            .

            if cycle_shares > 0 {

                 ....
                 ...
                 ..
                 .
    -----@>      } else {

                cycle.settled = true;
                self.s_redeem_cycles.write(_cycle_id, cycle);
                self
                    .emit(
                        RedeemCycleSettled {
                            cycle_id: _cycle_id,
                            cycle_type: cycle.cycle_type,
                            returned_amount: 0,
                            cycle_shares: 0,
                            cycle_rate: 0,
                            fee_collected: 0,
                        },
                    );
            }
        }
```

The assertion `cycle_shares > 0` guarantees that `cycle_shares` is always greater than zero before reaching the conditional statement. As a result, the condition `if cycle_shares > 0` will always evaluate to `true`, making the `else` branch logically unreachable.

Therefore, the `else` block is dead/unreachable code which can be safely removed to improve code clarity and maintainability.

Recommendation: The `else` block in the `redeem::_settle_redeem_batch_internal` function can be safely removed

Resolution: Fixed

[I-04] Possible rounding error in assets and shares calculation during mint

Severity: Information Risk

Description: Solidity vault rounds up for `epochAssets` and then calculate `epochShares` again from `epochAssets`:

```
@>         uint256 epochAssets = (epochShares * settlePrice + SHARE_PRICE_PRECISION - 1) /
SHARE_PRICE_PRECISION;
            if (epochAssets > pending) epochAssets = pending;
@>          epochShares = (epochAssets * SHARE_PRICE_PRECISION) / settlePrice;

            // Cap after round-trip: rounding up epochAssets can inflate epochShares above
            // sharesRemaining on low-decimal assets (e.g. USDC), causing underflow.
            if (epochShares > sharesRemaining) epochShares = sharesRemaining;

            assetsOut       += epochAssets;
            sharesRemaining -= epochShares;
            s_pendingDeposits[_controller][epoch] -= epochAssets;
            s_totalClaimableDeposits              -= epochAssets;
```

However, cairo counterpart vault doesn't recalculate `shares_to_take` from rounded-up `assets_for_shares`

```
let max_shares_here = pending * SHARE_PRICE_PRECISION / settlement_price;
                let shares_to_take = if max_shares_here <= remaining_shares {
                    max_shares_here
                } else {
                    remaining_shares
                };

                // Assets for these shares (round up to not over-mint); cap to pending
@>              let assets_for_shares = (shares_to_take * settlement_price + SHARE_PRICE_PRECISION
                    - 1)
                    / SHARE_PRICE_PRECISION;
                let consume = if assets_for_shares > pending {
                    pending
                } else {
                    assets_for_shares
                };

@>              total_shares += shares_to_take;
                total_consumed += consume;
                remaining_shares -= shares_to_take;
```

Impact:

Cairo vault consumes slightly more assets than solidity vault

Recommendation:

Like solidity vault, calculate `shares_to_take` from rounded-up `assets_for_shares`

Resolution: Fixed

[I-05] `s_min_redeem` can be enforced in `redeem`/`withdraw` functions to prevent users from holding dust amounts in `s_pending_redeems` and possible dust accumulation in pool

Severity: Information Risk

Description: Currently, `S_min_redeem` is enforced in the `request_redeem` and `request_fast_redeem` functions, but not in the `redeem` / `wuthdraw` functions.

This inconsistency allows a user to perform a partial redeem that may leave behind a residual share balance below `s_min_redeem`, resulting in dust amounts being held in `s_pending_redeems` for that user . When the user later attempts to redeem this remaining dust balance, the `gross_assets` calculation may round down to zero in `redeem` (particularly in cases where asset decimals are 6 and share decimals are 18, or during loss cycles) or revert in `withdraw` (because it roundsup in share calculation and this case can result in `shares <= locked_shares`)

As a result, the user effectively burns these residual shares without receiving any assets in return while using `redeem` or his transaction will always revert when using `withdraw` , while the corresponding asset value remains in the pool as unclaimable dust.

Impact:

Accumulation of dust assets in the pool.

Recommendation:

To mitigate this issue :

1. Add a check in `_redeem_internal` which checks if `locked_shares - _shares < s_min_redeem` and if true sets `_shares = locked_shares` to fully redeem the remaining balance and prevent users from holding sub-minimum (dust) share balances.

2. Add a similar check as above in `withdraw_internal`. If `locked_shares - _shares < s_min_redeem`, set `_shares = locked_shares` to fully clear the remaining balance.

when this condition is triggered:

- After calculating `gross_assets`, apply the fee using `sy_vault_lib::deduct_fee` instead of directly subtracting `_assets`.
- Finally, transfer the assets corresponding to `locked_shares` to the user instead of the value specified by him.

Resolution: Fixed

[I-06] `set_fast_redeem_fee_bps` allows 100% fee, leading to division by zero in `withdraw` and total fund loss in `redeem`

Severity: Information Risk

Description: The `set_fast_redeem_fee_bps` setter allows `fee_bps = 10000` (100%) via an inclusive upper-bound guard:

```
// vault.cairo:623-627
fn set_fast_redeem_fee_bps(ref self: ContractState, _bps: u256) {
    self.Base._assert_only_admin_gateway();
    assert(_bps <= BASIS_POINTS, SYVAULT_INVALID_PARAM);  // allows 10000
    self.Base.s_fast_redeem_fee_bps.write(_bps);
}
```

This creates two broken code paths for any fast-redeem cycle created while `fee_bps = 10000`:

`withdraw()` division by zero panic: `_withdraw_internal` calls `net_to_gross_assets(_assets, cycle.fee_bps)` which computes `denom = BASIS_POINTS - fee_bps = 0`:

```
// math.cairo:63-69
pub fn net_to_gross_assets(_net_assets: u256, _fee_bps: u256) -> u256 {
    if _fee_bps == 0 { return _net_assets; }
    let denom = BASIS_POINTS - _fee_bps;  // = 0 when fee_bps = 10000
    (_net_assets * BASIS_POINTS + denom - 1) / denom  // PANIC: division by zero
}
```

`redeem` total fund loss: `_redeem_internal` computes `fee = gross * 10000 / 10000 = gross`, so `net = gross - gross = 0`. The user receives 0 assets, but their shares were already burned during cycle settlement a total loss of redeemed value:

```
// redeem.cairo:389-396
let gross_assets = _shares * cycle.cycle_rate / SHARE_PRICE_PRECISION;
let (net_assets, fee) = if is_fast && cycle.fee_bps > 0 && !fee_whitelist {
    sy_vault_lib::deduct_fee(gross_assets, cycle.fee_bps)  // fee = gross, net = 0
} else {
    (gross_assets, 0_u256)
};
// ... transfers net_assets (0) to user
```

The `fee_bps` is snapshotted at cycle creation time in `_open_or_reuse_cycle`, so the impact persists for the lifetime of affected cycles even if the fee is later corrected.

Note: this same issue exists in the Solidity reference implementation (`syVault.sol:486-490`, `SyVaultRedeem.sol:285`, `SyVaultRedeem.sol:392`), where the guard also uses `>` instead of `>=`.

The protocol team confirmed that 100% fee has no legitimate use case. The agreed-upon maximum reasonable fee is 30% (3000 bps), and the setter should enforce this cap.

Impact:

The setter guard allows any value up to and including 10000 bps (100%). At extreme fee values:

- At `fee_bps = 10000` (100%): `withdraw` panics with division by zero, `redeem` returns 0 assets to users whose shares were already burned at settlement total loss of redeemed value.
- At any unreasonably high `fee_bps` (e.g., 9000 = 90%): users lose the vast majority of their redeemed value to fees. While technically functional, no legitimate protocol configuration would charge 90% on fast redeems.

The `fee_bps` is snapshotted at cycle creation time in `_open_or_reuse_cycle`, so the impact persists for the lifetime of affected cycles even if the fee is later corrected.

Recommendation:

Cap the fee at a reasonable maximum (3000 bps = 30%) as agreed with the protocol team:

```
+ const MAX_FAST_REDEEM_FEE_BPS: u256 = 3000;  // 30% maximum

  fn set_fast_redeem_fee_bps(ref self: ContractState, _bps: u256) {
      self.Base._assert_only_admin_gateway();
-     assert(_bps <= BASIS_POINTS, SYVAULT_INVALID_PARAM);
+     assert(_bps <= MAX_FAST_REDEEM_FEE_BPS, SYVAULT_INVALID_PARAM);
      self.Base.s_fast_redeem_fee_bps.write(_bps);
  }
```

Resolution: Fixed

[I-07] `MAX_DEPOSIT_CLAIM_BATCH` Can Cause Incomplete Share Minting

Severity: Information Risk

Description: The Cairo implementation introduces a `MAX_DEPOSIT_CLAIM_BATCH` constant (set to 100) that caps the number of deposit epochs processed per claim call. This limit is not present in the Solidity reference, where `_consumeDepositEpochsByAssets` and `_consumeDepositEpochsByShares` iterate over all settled epochs without restriction.

constants.cairo#L41

```
pub const MAX_DEPOSIT_CLAIM_BATCH: u64 = 100;
```

Both `_consume_deposit_epochs_by_assets` and `_consume_deposit_epochs_by_shares` compute a `batch_end` that silently stops iteration once 100 epochs have been visited:

deposit.cairo#L131-L135

```
let batch_end = if len - claim_idx > MAX_DEPOSIT_CLAIM_BATCH {
    claim_idx + MAX_DEPOSIT_CLAIM_BATCH
} else {
    len
};
```

When a controller has more than 100 unsettled deposit epochs queued, a mint call will process only the first 100 and return fewer shares than the caller requested, with no revert or indication that the claim was partial.

Impact:

A user who accumulated deposits across more than 100 epochs would receive fewer shares than expected from a single mint call. The remaining claimable deposits are not lost, they can be retrieved with follow-up calls, but the caller has no on-chain signal that the result was truncated. If the user faces gas issues from too many epochs, they can reduce the mint amount to lower the number of loop iterations instead.

Recommendation:

Remove `MAX_DEPOSIT_CLAIM_BATCH`.

Resolution: Fixed

[I-08] `claimable_fast_redeem_request` Returns An `_assets` Value That `withdraw` Cannot Accept

Severity: Information Risk

Description: `claimable_fast_redeem_request` computes the claimable net amount using floor rounding, while `_withdraw_internal` rounds the inverse conversion up. Because of this asymmetry, the number returned by the view cannot in general be passed back into `withdraw` without reverting.

The view computes:

```
gross_view  = floor(shares * cycle_rate / SHARE_PRICE_PRECISION)
fee_view    = floor(gross_view * fee_bps / BASIS_POINTS)
net_view    = gross_view - fee_view
```

`_withdraw_internal` takes `_assets` as the net the user wants and computes:

```
gross_needed  = ceil(_assets * BASIS_POINTS / (BASIS_POINTS - fee_bps))
shares_needed = ceil(gross_needed * SHARE_PRICE_PRECISION / cycle_rate)
assert shares_needed <= locked_shares
```

When `gross_view * fee_bps` is not a multiple of `BASIS_POINTS`, `net_to_gross_assets(net_view, fee_bps)` returns `gross_view + 1`. That extra unit forces `shares_needed` to exceed `locked_shares` and the call reverts with `SYVAULT_NO_CLAIMABLE_REDEEM`.

Cairo view, floor rounding:

```
fn claimable_fast_redeem_request(
    self: @ContractState, _request_id: u256, _controller: ContractAddress,
) -> u256 {
    let cycle = self.Redeem.s_redeem_cycles.read(_request_id);
    if cycle.cycle_type != CycleType::Fast || !cycle.settled {
        return 0;
    }
    let shares = self.Redeem.s_pending_redeems.read((_controller, _request_id));
    let gross = shares * cycle.cycle_rate / SHARE_PRICE_PRECISION;
    let (net, _) = if cycle.fee_bps > 0
        && !self.Base.s_fast_redeem_fee_whitelist.read(_controller) {
        sy_vault_lib::deduct_fee(gross, cycle.fee_bps)
    } else {
        (gross, 0_u256)
    };
    net
}
```

Cairo withdraw path, ceil rounding:

```
let mut gross_assets = if has_fee {
    sy_vault_lib::net_to_gross_assets(_assets, cycle.fee_bps)
} else {
    _assets
};

let mut shares = if cycle.cycle_rate > 0 {
    (gross_assets * SHARE_PRICE_PRECISION + cycle.cycle_rate - 1) / cycle.cycle_rate
} else {
    0_u256
};
assert(shares > 0 && shares <= locked_shares, SYVAULT_NO_CLAIMABLE_REDEEM);
```

Impact:

EIP-7540 and EIP-4626 require that a user can always withdraw up to the value reported by the claimable view. Here `claimable_fast_redeem_request` advertises a maximum that `withdraw` rejects in almost every non-trivial case. Integrations that forward the view output directly to `withdraw` will fail, and users following frontend hints will see their transactions revert. The shares remain claimable through `redeem`, so no funds are lost, but the claimable view effectively overstates what the exact-fill `withdraw` path can honor.

Proof of Concept:

With `fee_bps = 3000` (30%) and `locked_shares` such that `gross_view = 101`:

1. `fee_view = floor(101 * 3000 / 10000) = 30`

2. `net_view = 101 - 30 = 71`

3. User reads `71` from `claimable_fast_redeem_request` and calls `withdraw` with `_assets = 71`

4. `_withdraw_internal` computes `gross_needed = ceil(71 * 10000 / 7000) = 102`

5. `shares_needed = ceil(102 * SHARE_PRICE_PRECISION / cycle_rate) > locked_shares`

6. Call reverts with `SYVAULT_NO_CLAIMABLE_REDEEM`

Recommendation:

Align the view with the rounding used by `withdraw` so that the advertised value is always reachable. The simplest correct fix is to compute the same `gross_view` the current view produces and then subtract the fee through the same inverse path used by `_withdraw_internal`, i.e. return the largest `_assets` such that `net_to_gross_assets(_assets, fee_bps)` is less than or equal to `gross_view`:

```
let gross = shares * cycle.cycle_rate / SHARE_PRICE_PRECISION;
let net = if cycle.fee_bps > 0 && !whitelist {
    // Largest net such that net_to_gross(net) <= gross.
    (gross * (BASIS_POINTS - cycle.fee_bps)) / BASIS_POINTS
} else {
    gross
};
```

An equivalent and smaller change is to return `net - 1` whenever a fast fee is applied and `net > 0`, which guarantees reachability at the cost of one wei of precision.

Resolution: Fixed

Gas optimization

[G-01] Redundant Storage Read in Deposit Claim Loop

Severity: Gas Optimization

Description: In `_consume_deposit_epochs_by_assets` and `_consume_deposit_epochs_by_shares`, after writing the updated `s_pending_deposits` value via `storage.write`, the code immediately re-reads the same slot via `storage.read` to check if it equals zero. Since the value just written is `pending - consume`, this is already known locally and the storage read is redundant.

Impact:

Each unnecessary `storage.read` costs extra gas on every loop iteration that reaches the drain check. For controllers with many settled epochs, claiming deposits iterates through multiple entries, compounding the wasted gas.

Proof Of Concept:

_consume_deposit_epochs_by_assets

```
self.s_pending_deposits.write(key, pending - consume);

// Redundant storage read - value is already known as `pending - consume`
if self.s_pending_deposits.read(key) == 0 && i == claim_idx {
    claim_idx += 1;
}
```

_consume_deposit_epochs_by_shares

```
self.s_pending_deposits.write(key, pending - consume);

if self.s_pending_deposits.read(key) == 0 && i == claim_idx {
    claim_idx += 1;
}
```

Recommendation:

Replace the storage read with a local variable comparison in both functions:

```
let new_pending = pending - consume;
self.s_pending_deposits.write(key, new_pending);

if new_pending == 0 && i == claim_idx {
    claim_idx += 1;
}
```

Resolution: Fixed